

arXiv 2609.11148 · PVLDB 2026 · **Best Industry Paper**

# OmniTable

A unified wide table for petabyte-scale LLM data curation — and what it says about separating *logical structure* from *physical layout* in a data lake.

- logical unification
- physical separation
- wide table as virtual view
- Catalog-driven

# The paper

- **19 authors, all at Ant Group** — corresponding author Jun Zhou
- PVLDB Vol. 19, No. 12, pp. 4276–4289, 2026 · doi 10.14778/3827998.3828032
- **VLDB 2026 Best Industry Paper** (per the arXiv comment field)
- Self-described “**architecture blueprint**” — design and production measurements, not a new algorithm
- Runs in production on **35 PB**, 305B+ records, 392+ batches, 16 physical tables
- Headline: curation cycle **14 days → 2.5 days (5.6×)**, manual steps –73%

# Where lakehouse formats stop

- Delta Lake / Iceberg / Paimon already give you ACID, time travel and schema evolution on object storage.
- They remain **table-centric**: the table is simultaneously the logical unit and the physical unit.
- Two things they do not treat as first-class lifecycle objects:
  - a **column / UDF dependency DAG**
  - **operator-aware CPU/GPU routing**
- In LLM curation the unit of iteration is a **feature**, not a table — columns are computed by UDF DAGs, not appended as rows.

OmniTable positions itself as a reusable layer *above* storage, not a replacement for it.

# The pipeline maze

## Data silos

Dozens of sources scattered over **hundreds of physical tables**. Cross-dataset discovery is the hard part.

## Costly features

For a *single* feature, an engineer had to “**drag and drop 106 tables onto the task canvas.**”

## Broken lineage

UDF logic spread across codebases, no centralized version control. Definitions drift; iterations can't be traced.

The iteration unit is a *table*, so cost scales with the number of datasets — the exact opposite of what ablation-driven research needs.

# Four contributions

1. **Unified wide-table abstraction** — multi-source data and thousands of derived features under one logical schema, via logical–physical mapping.
2. **Declarative feature lifecycle** — dependency resolution, execution planning, operator fusion and lineage; “*declare-and-execute*” replaces manual orchestration.
3. **Adaptive execution + autonomous governance** — CPU/GPU routing, adaptive tuning, UDF-level fault tolerance, background storage-layout optimization.
4. **Hybrid-accelerated exploration** — global ID index, transparent OLAP offload, background materialized views.

the whole design, in five lines

# Logical Unification, Physical Separation

- Users see **one** wide table. The system stores **many**.
- The logical schema is a **contract, not a constraint**.
- A **Catalog** owns the mapping and is the single consistency decision point.

Cost: ~8–15% extra storage for hot column groups. Payoff: 5.6× end-to-end, –73% manual steps.

# The wide table is a virtual view

## What users see

### 1 logical table.

Unified schema, 800+ columns, plain SQL. No physical layout visible.

## The Catalog

### The only thing in the middle.

Owens logical→physical mappings, versions, batch registry.

## What exists

**4–6 physical tables** (a *Table Family*): different column subsets, batch-organized, engine-specific.

- The logical table has **no physical existence** — it is resolved per query.
- Nobody writes to it: ingestion adds *rows*, backfill adds *columns*, both into physical tables.

# The logical schema

```
CREATE TABLE OmniTable (  
  _ai_unique_id_  STRING NOT NULL,  -- global primary key & lineage anchor  
  _ai_append_name_ STRING NOT NULL,  -- ingestion batch / source tag  
  
  RawData          STRUCT<...>,      -- stage-aware core column groups  
  ProcessedData     STRUCT<...>,      --  
  TrainableData     STRUCT<...>,      --  
  
  Feature_1 <Type1>, ... Feature_N <TypeN>  -- unbounded feature columns  
);
```

Illustrative (paper's Listing 1). This is a **logical contract** — there is no physical `CREATE TABLE` behind it. The Catalog maintains the mapping that makes it real.



# Four principles of the logical model

- **Global primary key alignment** — one key aligns rows across sources, stages and features, enabling backfill joins, point lookups and lineage auditing.
- **Stage-aware data evolution** — RawData / ProcessedData / TrainableData, with optional version suffixes (ProcessedData\_v2) so processing iterations become versioned columns.
- **Unbounded feature column expansion** — quality scores, domain labels, compliance flags, dedup signatures — registered, versioned and refreshed as columns.
- **Batch-level governance** — the batch is the basic unit for auditing, task splitting and query pruning.

# Three columns that carry the design

## `_ai_unique_id_`

Global PK. Default is **MD5 of raw\_data** — a deterministic content hash, so dedup is free and needs no coordination. Random UUID when content dedup is unwanted (SFT).

## `_ai_append_name_`

Batch + source + version. One tag doing three jobs: **backfill scheduling granularity, query pruning key, audit trail.**

## The stage groups

RawData → ProcessedData → TrainableData: original payload, cleaned intermediate form, trainable form.

Because the key is a content hash, identical content from different sources *collapses into one row*. That is intentional — but it is a semantic choice about provenance, not a storage detail.

# The Table Family — five layers, one boundary

## Logical side (semantics)

- `LogicalTable` — the user-facing wide table, e.g. `aidata://tables/web`
- `LogicalColumn` — name, type, **analysis type**, comments

## Physical side (layout)

- `PhysicalTableGroup` — groups tables by function or batch; **this is the lock granularity**
- `PhysicalTable` — on MaxCompute or OSS: path, partitions, file format
- `PhysicalColumn`

The boundary is exactly **one layer of indirection**: `LogicalColumn` ↔ `PhysicalColumn`, held by **bidirectional references** in the Catalog. Everything above is semantics; everything below is layout.

# What a mapping entry carries

```
LogicalColumn: quality_score    (type: double, analysisType: score)
```

```
    physical location -> PhysicalTable: web_main      (engine, path, partitions, format)
```

```
    version           -> feature version, not schema version
```

```
    updated_at        -> timestamp
```

```
...and the reverse reference, so a PhysicalColumn knows which  
    LogicalColumn it serves.
```

- **Version + timestamp on every entry** → historical backtracking is free.
- Metadata lives in a **relational DB with optimistic locking** for concurrent updates.
- Production metadata query latency stays at the **millisecond level** — the indirection is cheap on the read path.

# The batch dimension

Every ingestion atomically registers an **Append entry** in the Catalog:

```
Append {  
  _ai_append_name_: "cc"           // batch name  
  physicalTables:   [web_main, web_colshard_2, ...]  
  rowCount:        ...  
  ingestedAt:       ...  
  source:           ...  
}
```

- `WHERE _ai_append_name_ = 'cc'` lets the optimizer **locate the exact physical table subset** and skip everything else — I/O reduction with no data movement.
- The same identifier is the scheduling unit for incremental backfill and the unit for audit backtracking.

# Resolving a query against a table that doesn't exist

- Logical columns may live in **different physical tables** (because of column splitting, or batch organization).
- So translation is **not a table-name substitution**. It is multi-step reasoning:
  1. Parse the SQL, extract referenced columns and predicates.
  2. Ask the Catalog for **each column's physical location**.
  3. Build a plan — typically a **multi-table JOIN on `_ai_unique_id_` with UNION ALL across batches**.
  4. Apply rewrites: predicate pushdown, column pruning, batch pruning.
  5. Try to match an acceleration path: ID index / OLAP offload / materialized view.

## The rewrite, concretely

```
-- what you write: one logical table
SELECT _ai_unique_id_, quality_score, math_recall
FROM   adata://tables/web
WHERE  _ai_append_name_ IN ('cc','cc2') AND quality_score > 0.7;

-- what the optimizer builds: JOIN on the global key, UNION ALL across batches
SELECT a._ai_unique_id_, a.quality_score, b.math_recall
FROM   ( SELECT * FROM main_cc  WHERE quality_score > 0.7
        UNION ALL
        SELECT * FROM main_cc2 WHERE quality_score > 0.7 ) a
JOIN   ( SELECT _ai_unique_id_, math_recall FROM colshard_2 ) b
ON     a._ai_unique_id_ = b._ai_unique_id_;
```

Illustrative reconstruction — the paper describes this rewrite but does not print the SQL. **Column pruning** is what stops the large `raw_data` column from ever being read.

# Column splitting: crossing the engine's width wall

- The web wide table has **800+ logical columns** — but MaxCompute imposes a **~1200-column** physical limit.
- The governance service groups columns by **access frequency from query logs**.
- Low-frequency feature columns migrate to auxiliary tables (**Column Shards**) containing only `_ai_unique_id_` plus the migrated columns.
- The main table keeps high-frequency and core columns.

In production: **800+ logical columns distributed across 4–6 physical tables of 200–300 columns each**. Queries touching only high-frequency columns need *no JOIN at all*.



# What column splitting actually changes

BEFORE    one physical table is close to the ~1200-column engine limit

[all 800+ LogicalColumns] --> PhysicalTable: web\_main

AFTER     only mapping pointers change; the logical DDL is untouched

quality\_score        (hot) --> PhysicalTable: web\_main

lang\_detect         (hot) --> PhysicalTable: web\_main

math\_recall\_v4       (cold) --> PhysicalTable: web\_colshard\_2    [id + col]

boilerplate\_flag     (cold) --> PhysicalTable: web\_colshard\_3    [id + col]

Physical table names are illustrative. The load-bearing claim: **no logical schema change, no consumer change** — the optimizer simply reads the latest mapping. Row splitting and small-partition merges work the same way.

# The invariant that makes it work

*Logical view stability is decoupled from physical layout evolvability.*

- Frontend services **read** mappings to build plans. Backend services **write** optimization results back.
- Neither knows about the other. The Catalog is the only meeting point.
- Physical changes are **fully transparent** because the optimizer always fetches the current mapping.
- Consequence: the system can reorganize storage *forever* without anyone's SQL changing.

# A virtual view needs a commit protocol

- Continuous writes at PB/day plus growing column counts inevitably degrade: small files, column limits, partition skew.
- So the system continuously **rewrites physical layout in the background** — while queries and ingestion keep running.
- That means a reader can be mid-query when its physical tables are being split or merged.
- The answer is not locking readers. It is **making layout changes invisible until an atomic commit**.

# Prepare – Execute – Commit

## Prepare

Acquire an exclusive lock at **PhysicalTableGroup** granularity. If a frontend write holds the target, the optimization is simply **postponed**.

## Execute

Write the reorganized data to a **staging area**. Existing files are never modified in place.

## Commit

**Atomically** register new files, update mappings, mark old files for reclamation. Failure → metadata rollback + staging cleanup.

The new layout becomes visible **only after a successful commit**. Subsequent queries then benefit automatically — with no coordination.

# Reads don't lock — they snapshot

- Frontend reads **do not take exclusive locks**.
- Instead they take a **consistent metadata snapshot at query start**.
- In-flight queries are therefore unaffected by concurrent background reorganization.
- The lock is only needed where writes collide — at `PhysicalTableGroup` granularity, which is coarse enough to be rare and fine enough to be parallel.

This is the piece that lets a virtual view be rewritten underneath live traffic: snapshot isolation on *metadata*, staged writes on *data*, atomic pointer swap to publish.

## What it runs on

- **web** — 300B+ records, 25 PB, 800+ logical columns, **6 physical tables**, 200+ batches, 13 sources
- **code** — 3.4B / 3.8 PB / 350 columns / 4 tables / 85 batches
- **pdf** — 1.8B / 5.2 PB / 280 columns / 3 tables / 62 batches
- **post\_sft** — 210M / 0.8 PB / 120 columns / 3 tables / 45 batches
- **Total 305B+ records, 35 PB, 16 physical tables, 392+ batches**

Storage is Parquet on OSS plus a MaxCompute warehouse; Catalog on MySQL, index on HBase, OLAP on ClickHouse. A real production deployment, not a testbed.

# The two curves that validate the separation

Data volume: 1 TB → 25 PB

- **OmniTable:** stable **18–23 TB/h**
- No governance: ~10 TB/h at 5 PB, ~5 TB/h at 25 PB
- Legacy: ~2 TB/h at 25 PB (**9.5× slower**)

Schema width: 200 → 2500 columns

- **OmniTable:** P95 **25 s → 38 s (1.53×)**
- No governance: 110 s at 1500 columns (~3×)
- Beyond 1200 columns, the ablation's queries **fail outright**

The width result is the key one for a data lake: **it crosses a hard engine limit with no performance discontinuity** — because crossing it is a mapping update, not a table redesign.

# What to take

- **One layer of indirection is the entire design.** A single `LogicalColumn` ↔ `PhysicalColumn` boundary, owned by one service, buys both schema evolution and layout freedom.
- **Treat engine limits as architecture constraints, not user problems.** 1200 columns is a physical fact; it should never reach the person writing SQL.
- **Make the batch a first-class dimension.** One identifier serving scheduling, pruning and audit is unusually good design economy.
- **Give the virtual view a commit protocol.** Stage writes, snapshot reads, atomic publish.
- **Adoption beats capability.** The logical table speaks plain SQL and lineage records are queryable tables — value first, new workflow second.



# What to discount

- **5.6× is workflow-level, not system-level.** The baseline is historical task logs measuring coordination cost (locate tables, wire pipelines, diagnose, resubmit). It is explicitly orthogonal to compute-level optimization — do not read it as a throughput comparison against another engine.
- **No public benchmark and no head-to-head system comparison.** The authors' reason (settings too different for a shared benchmark) is fair, but the numbers are not independently reproducible.
- **"Second-level point lookups" are 5–15 s (P50 8.3 s)** — an index-assisted partial scan plus multi-table column assembly, not a point read.
- **Key hyperparameters are missing** — materialization thresholds and the column-split access-frequency cutoff are only ever "exceeds a threshold."

# Takeaways for our own lake

- If logical and physical are the same object today, **a single mapping layer is the cheapest first step** — you get schema evolution without touching consumers.
- **Budget the indirection.** It cost them ~8–15% extra storage for hot column groups, and 5–8% of cluster resources for continuous compaction.
- **Decide the global key up front.** MD5-of-content buys free dedup but collapses provenance — a semantic decision, not a storage one.
- **Reorganization needs staging + atomic commit + reclamation**, and a lock granularity coarse enough to be rare, fine enough to be parallel.

Full analysis: `my-wiki/05-References/0mniTable` 论文总结（中文）.md and the English counterpart.